

Please ignore the assesment links provided in between the notes

✓ Content

N Layer-1

- **Recap**
- **Backpropagation for single layer**
 - Chain rule
 - Scratch code for backprop
- **MLP: Multi layer Preceptron**
 - Nlayer NN with 3 outputs
 - how **hidden** layers are useful
- Assessments:
 - [Trainable parameters](#)
 - [Number of connections](#)
 - [How many parameters?](#)
 - [Convergence](#)
 - [Complete the updating code](#)

N Layer-2

- **Activation Functions**
 - vanishing gradient issue
 - ReLU and Leaky ReLU as solution to vanishing gradient
- Assessments
 - [Tanh and Leaky ReLu](#)
 - [Dog and cat classifier](#)
 - [Maximum value of derivates](#)
 - [Rectified Linear Unit](#)
 - [No Activation Function](#)
 - [ReLu\(vanishing/ exploding gradient\)](#)
- **Notations**
 - N layer notations
- Assessments

- Fully Connected Neural Network
- Compare the learnings
- **Forward Propagation for multi-layer**
 - Code for Forward propagation in multi-layer NN
- Assessments
 - Hidden layer
 - Intercept and Bias

✓ Quizzes

✓ Quizes of lec-1

Quiz-1

What happens if we set the activation function of a neuron ($WX+b$) as sigmoid?

- a) It is not a neuron anymore
- b) We get a LRU
- c) We don't have to add a bias anymore
- d) We get a perceptron
- e) It does not add any non-linearity

Ans: b

- If we set the activation function of a neuron as sigmoid, it is still a neuron
- It is called a Logistic Regression Unit (LRU), as that neuron becomes identical to a Logistic Regression model, in functionality.
- We still need to add a bias to a LRU, just as in LogReg
- Perceptron employs step function as its activation function, it is different from a LRU.
- Sigmoid function does add a layer of non-linearity. Recall its graph. This non-linearity is what helps build complex feature relations.

Quiz-2

Which of the following can be represented as a single-neuron model?

- a) Linear Regression
- b) SVM
- c) Logistic Regression
- d) KNN

Ans: a, b, c

If the activation function is set as sigmoid function -> Logistic Regression

If the activation function is set as hinge loss function -> Linear SVM model

Quiz-3

The number of features for input vector are 2.

Suppose we are implementing log. regression using neuron. How many parameters will neuron

- a. 1
- b. 2
- c. 3
- d. 4

Ans: 3

Explanation:

Since we have 2 features, there will be one weight corresponding to each feature i.e. w_1 and w_2 .

- and there'll be one param corresponding to bias (b)

Hence, total of 3 params

Quiz-4

How many total weights and bias there will be between 2 feature input and 3 neurons ?

- a. 3
- b. 6
- c. 9
- d. 12

Ans. $6 + 3 = 9$.

Since there are 2 inputs, and 3 neurons, there would be a weight associated with each pair of input and neuron.

Hence, weights = $2 \times 3 = 6$

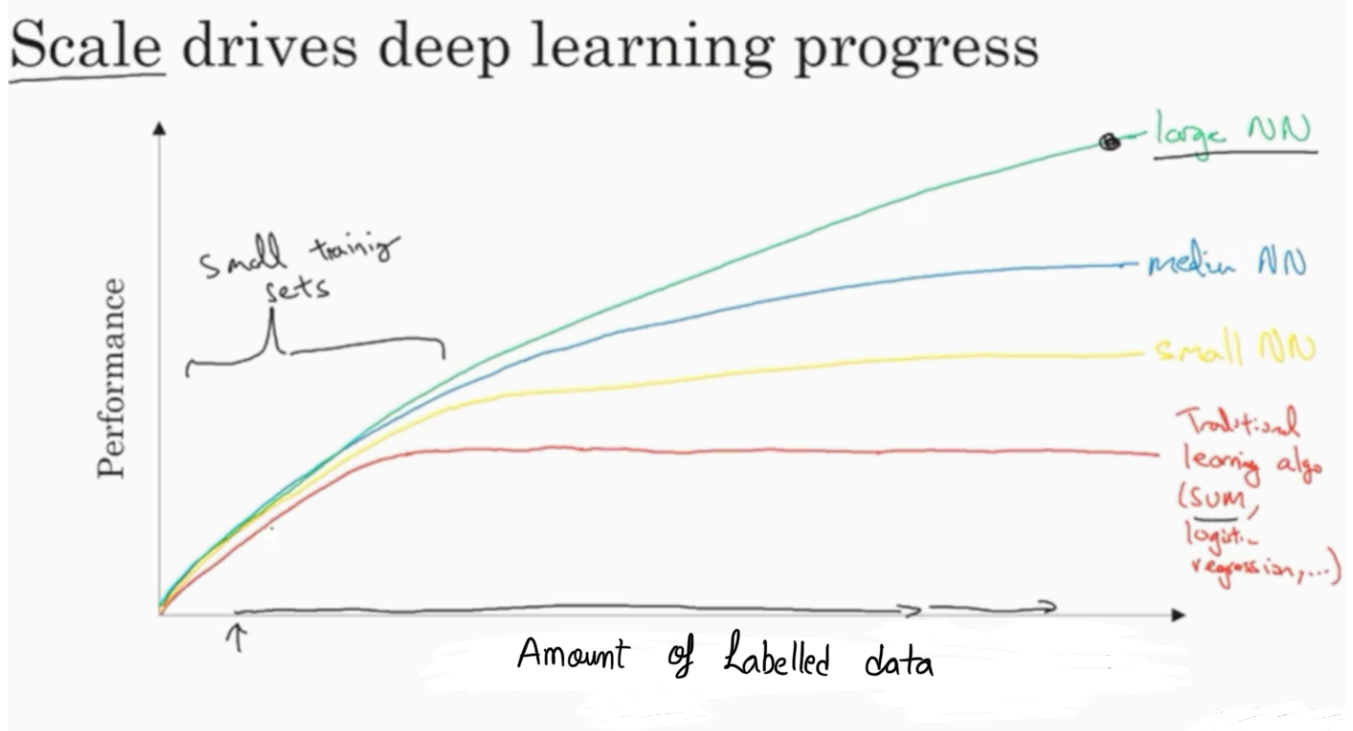
And each neuron has bias associated with it i.e 3 biases Hence, **total weights and biases = $6 + 3 = 9$**

Quiz-5

If we have a small training data set, which of the following will perform the best?

- a) Traditional ML Algos: SVM, LogReg, etc
- b) Small NN network
- c) Medium NN network
- d) Large NN network
- e) All of may give comparable results

Ans: e



Source: Andrew Ng

✓ Quizes of Lec-2

Quiz-1

Which of the following would we employ for a multi-class classification setting?

- a) sigmoid function as activation function
- b) cross entropy as loss function
- c) softmax function as activation function
- d) log loss as loss function

Ans: b, c

- In binary setting
 - we use Sigmoid for probability
 - and Binary CE/logloss as loss function
- In multiclass setting,
 - we use softmax to compute class probabilities
 - and Cross entropy as loss function

Quiz-2

Consider the following variables:

d -> dimensionality, number of features

n -> number of classes

m -> number of training examples

What will be the shape of the weights matrix?

- a) $m \times n$
- b) $m \times d$
- c) $n \times d$
- d) $d \times n$

Ans: d

In order to get the output matrix ($m \times n$), as a dot product of data matrix ($m \times d$) and weight matrices, the shape of weight matrix has to be: **$d \times n$**

$$w^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \end{bmatrix}_{d \times n}$$

Handwritten diagram illustrating matrix multiplication for a neural network layer. It shows a data matrix of size $m \times d$ (labeled 'data' and ' $m \times d$ ') multiplied by a weight matrix of size $d \times n$ (labeled ' $d \times n$ '). The result is an output matrix of size $m \times n$ (labeled ' $m \times n$ ').

Quiz-3

What will be the shape of bias matrix?

- a) $1 \times d$: because there are d features and it can get broadcasted
- b) $m \times d$: because there are d features and m train examples
- c) $1 \times n$: because there are n neurons, and this will get broadcasted
- d) $m \times n$: because $X.W$ will fetch a $m \times n$ matrix

Ans: c

- The bias term has to be added to the dot product of $X.W$.
- Therefore, the shape of bias matrix should also be: **$m \times n$**

However, the value of bias will not change for different data points.

- The same bias values can be broadcasted for m examples.
- Therefore, the shape of bias matrix becomes: **$1 \times n$**

Quiz-4

Backpropagation is used to calculate gradients, in order to update the values of of __ .

- a) weights
- b) biases
- c) both
- d) None of above

Ans: c Both weight and bias values are updated using backprop algorithm as:

$$w_i = w_i - lr * \frac{\partial J}{\partial w_i}$$

$$b_i = b_i - lr * \frac{\partial J}{\partial b_i}$$

✓ Quizes of lec-3 (To be moved in other script)

Quiz-1 and 2

Suppose we have a MLP network that has 2 inputs going into 3 neurons in the first layer. The results of these neurons are going to 5 neurons in the **hidden** layer. The output layer consists of 2 neurons.

What will be the shape of W^2

- a) 1×3
- b) 2×3

- c) 5×2
- d) 3×5

Ans: d

W^2 refers to the weight matrix connecting the first layer to the second layer.

There are 3 neurons in L-1, and 5 in L-2. Therefore, there will be a total of 15 weight values, and the shape of W^2 will be 3×5

What will be the shape of b^3 ?

- a) 1×2
- b) 2×1
- c) 5×2
- d) 2×5

Ans: a

Since there are 2 neurons in layer 3, i.e. output layer, we will have 2 biases.

This will be represented as 1×2 matrix.

Quiz-3

Can we use a NN for a regression task?

- a) No.
- b) Yes, by adding more layers in order to increase computation.
- c) Yes, by using linear functions for activation throughout the network.
- d) Yes, by using linear function for activation for the neuron(s) in output layer.

Ans: d

If the last neuron in chain is:

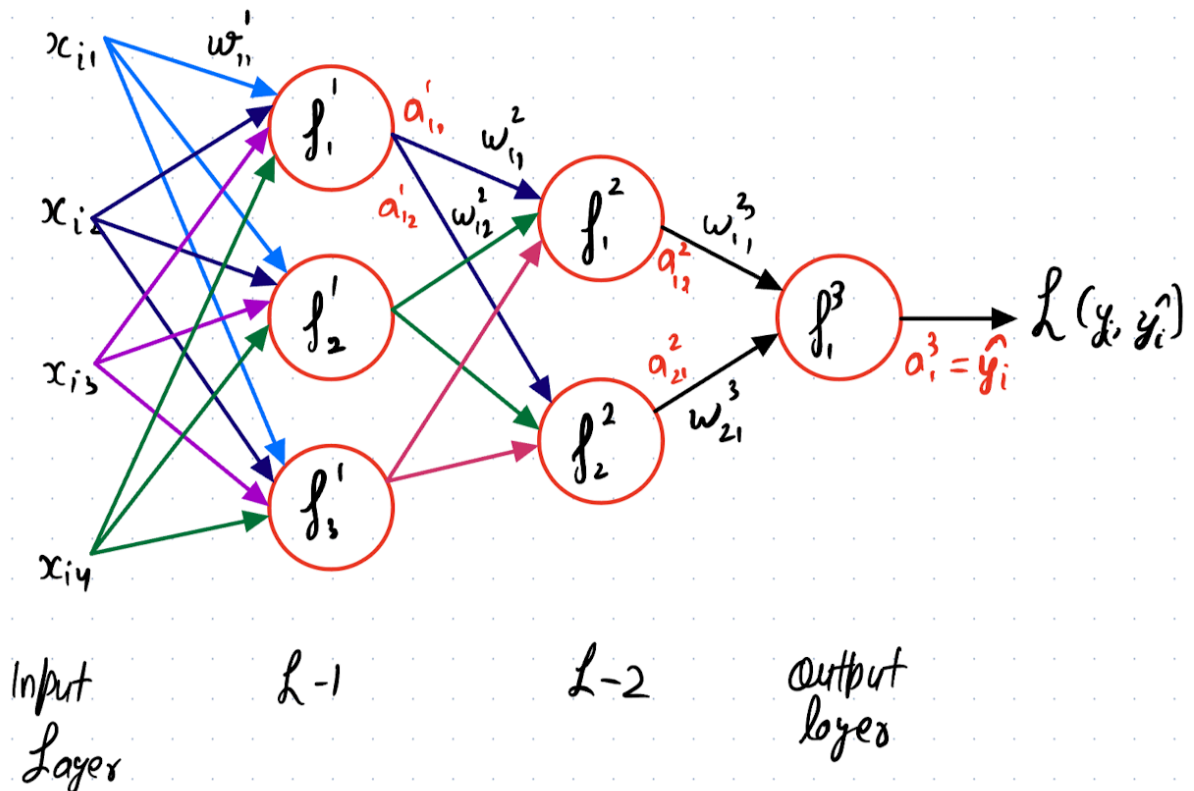
- Linear --> then NeuralNet will do regression. Ex: $y = x$
- Probabilistic (Sigmoid/softmax) --> NeuralNet will do classification. Ex: $y = \text{sig}(x)$

If the activations of intermediate layers are linear also, then, NN wouldn't be able to create high order complex features

The activations in intermediate layers should always be non-linear.

Quiz-4

Consider the following NN:-



How will we calculate the value of $\frac{\partial Loss}{\partial w_{11}^1}$?

- a) $\frac{\partial L}{\partial a_1^3} \cdot \frac{\partial a_1^3}{\partial a_{11}^2} \cdot \frac{\partial a_{11}^2}{\partial a_{11}^1} \cdot \frac{\partial a_{11}^1}{\partial w_{11}^1}$
- b) $\frac{\partial L}{\partial a_1^3} \cdot \frac{\partial a_1^3}{\partial a_{21}^2} \cdot \frac{\partial a_{21}^2}{\partial a_{12}^1} \cdot \frac{\partial a_{12}^1}{\partial w_{11}^1}$
- c) $\frac{\partial L}{\partial a_1^3} \cdot \frac{\partial a_1^3}{\partial a_{11}^2} \cdot \frac{\partial a_{11}^2}{\partial a_{11}^1} \cdot \frac{\partial a_{11}^1}{\partial w_{11}^1} + \frac{\partial L}{\partial a_1^3} \cdot \frac{\partial a_1^3}{\partial a_{21}^2} \cdot \frac{\partial a_{21}^2}{\partial a_{12}^1} \cdot \frac{\partial a_{12}^1}{\partial w_{11}^1}$
- d) $\frac{\partial L}{\partial a_1^3} \cdot \frac{\partial a_1^3}{\partial a_{11}^2} \cdot \frac{\partial a_{11}^2}{\partial a_{11}^1} \cdot \frac{\partial a_{11}^1}{\partial w_{11}^1} \cdot \frac{\partial L}{\partial a_1^3} \cdot \frac{\partial a_1^3}{\partial a_{21}^2} \cdot \frac{\partial a_{21}^2}{\partial a_{12}^1} \cdot \frac{\partial a_{12}^1}{\partial w_{11}^1}$

Ans: c

As per the backpropagation, we employ chain rule to calculate the gradient of each parameter, in order to update them.

There can be multiple paths to reach a certain parameter.

In order to combine the derivative from path-1 and path-2, we take the **sum** of the derivatives in these paths.

Quiz-5

Which of the following is true regarding Vanishing gradients?

- a) This is caused when we're using sigmoid or tanh as activation functions
- b) This is caused when we're using ReLu or Leaky ReLu as activation functions
- c) This happens because the gradients are ~ 0 , for most of the values of z , causing the learning to be too slow
- d) This happens because the learning is too fast, causing it to overshoot, and not find the minimum

Ans: a, c

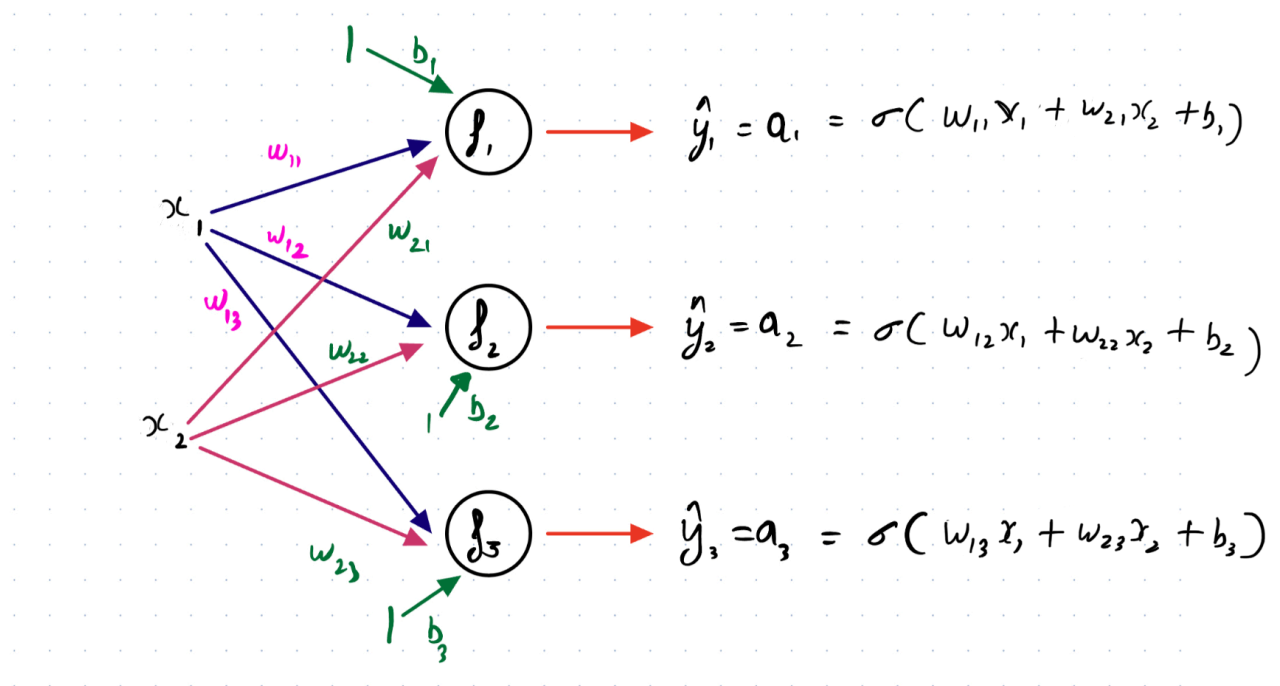
For sigmoid and tanh, the derivative is very close to zero for most values.

This makes the gradients ~ 0 , thereby causing the learning to be too slow.

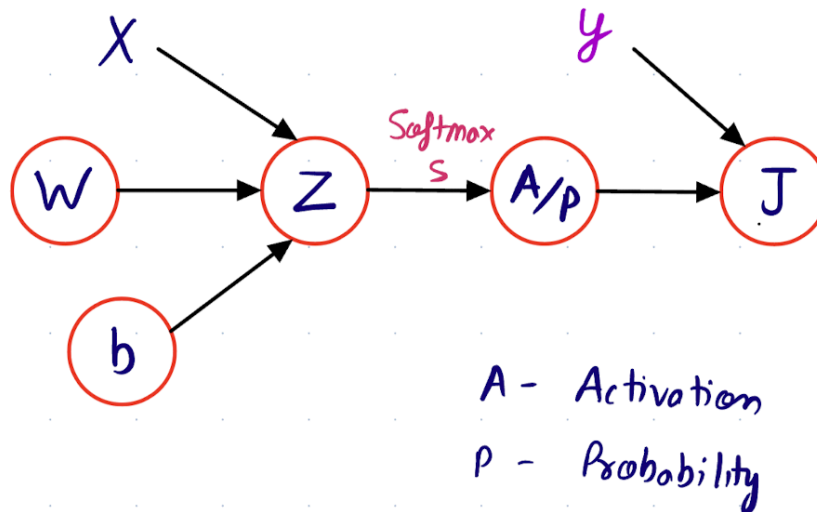
Double-click (or enter) to edit

✓ Backward propagation

We had the following NN for our multiclass classification



Let's draw computational graph for this



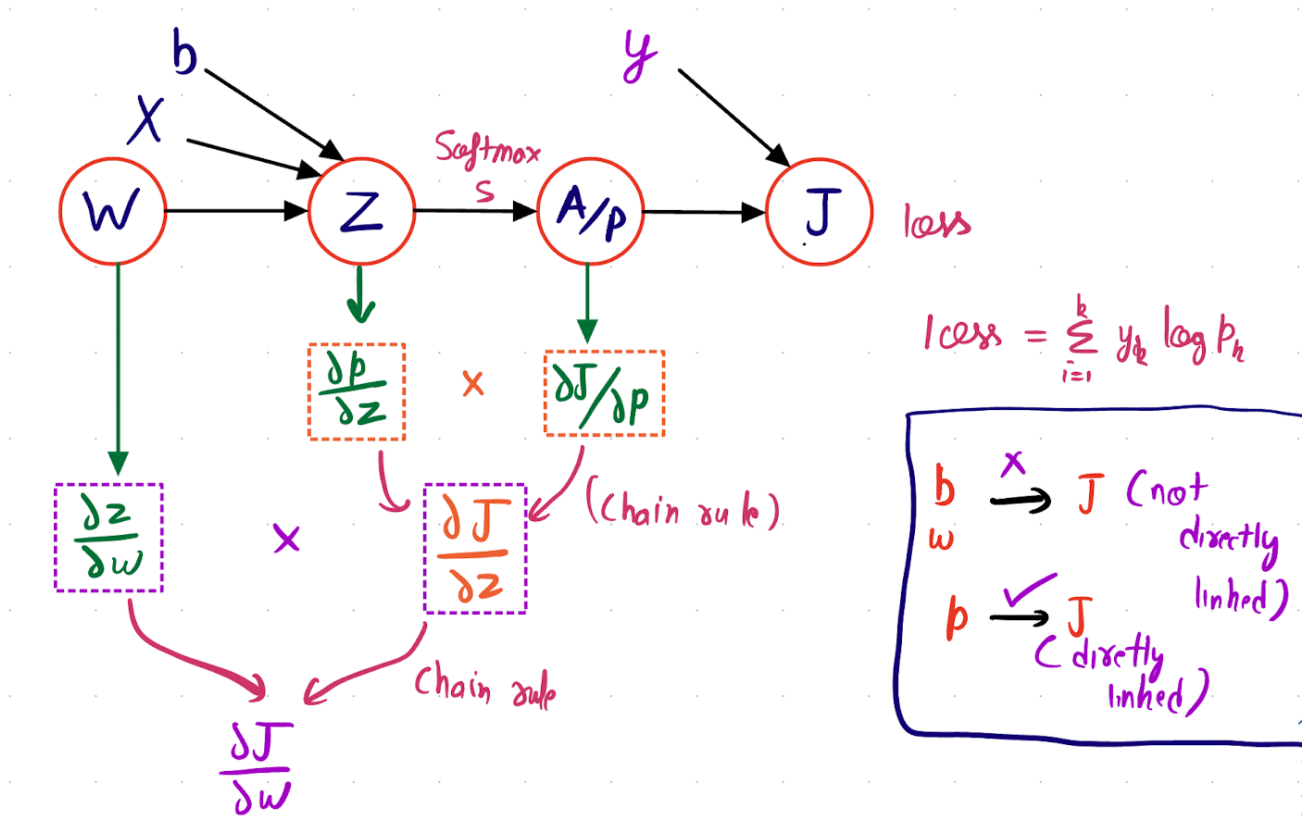
✓ Question: How did we decrease the loss in Logistic Regression?

- We calculate partial derivative of J wrt each W and b .
- And then we iteratively updated the params values of W and b
- What does the partial derivative explain intuitively? How does J change with a small change in any param?

Problem: **We cannot directly calculate dW and db here**

- But what we can see is the error J is directly dependent on p , so we can calculate $\frac{\partial J}{\partial p_i}$ directly
- Further, the p in-turn depend on z , and z is of course dependent on W (and b).

Lets break this down using chain rule



$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial z} \frac{\partial z}{\partial w}$$

We can see from the picture that $\frac{\partial J}{\partial z}$ can be calculated using

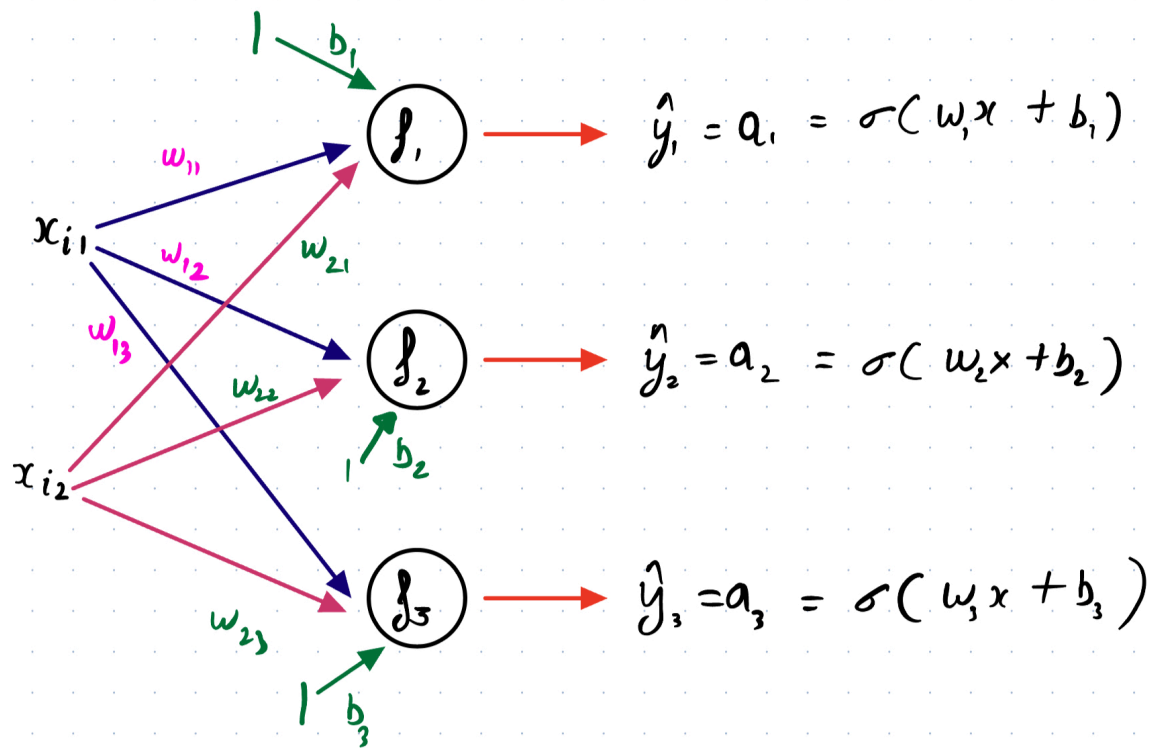
$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial p} \frac{\partial p}{\partial z} \frac{\partial z}{\partial w}$$

- Since, J (cost function) is the final function we want to optimise, we would come across a lot of intermediate calculations like $\frac{\partial J}{\partial \langle \text{var} \rangle}$.
- In Python, we will represent gradients of J , $\frac{\partial J}{\partial w}$ as dw .
- And the equation $\frac{\partial J}{\partial w} = \frac{\partial J}{\partial z} \frac{\partial z}{\partial w}$ can be represented as

$$dw = dz \frac{\partial z}{\partial w}$$

Since, we are moving from right to left across the computational graph, we call this process backpropagation.

We made a simple Neural network to classify a multiclass setting use case



(Revision) Question: How many weights did we have for 2 feature input and 3 neurons?

Ans: Since we have 2 input and 3 neuron, we have total of $2 \times 3 = 6$ weights and 3 bias (one per neuron.)

(Revision) What activation did we use in output ?

Ans: Softmax

Now, we only had a simple network of 3 neuron.

What if we add more neuron to our network and make it more complex ?

Let's see how we do it

✓ Let's calculate gradients using backpropagation

- We previously calculated $\frac{\partial z}{\partial w}$ in LR - since $z = w^T x$, $\frac{\partial z}{\partial w} = x$
- Now, we need to calculate dz , $dz = \frac{\partial J}{\partial p} \frac{\partial p}{\partial z}$

Turns out that the result for dz has a very neat and intuitive solution (won't get into derivation)

$$dz = (p_k - y_i)$$

- First, the second term is called an **Indicator Function**,
 - where p_k is the prob. of class K
 - y_i is 1 for the ground truth label, else 0.
 - e.g if $y_i = 2$ and probs = $[0.2, 0.3, 0.5]$
 - As the second index of the probs is the one with the correct class
 - $dz = [0.2-0, 0.3-1, 0.5-0] = [0.2, -0.7, 0.5]$

Note: If we were saving ground truth, y , as one-hot encoded vectors,

- We could simply subtract the probabilities with the one-hot encoded vector

We won't go into its derivation, but lets intuitively understand this

Why are we subtracting 1 only from the probability of true class?

- The use of subtracting 1 from the true class is to make it derivative negative.
- For the true class (middle), increasing its probability values will decrease the error (0.3 --> -0.7) - aligns with goal.
- For other values, first and last, increase (or keeping it same) their probability values will increase the error - aligns with goal.

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

```
!gdown 1dL0Pwh01o3k8p_hK633ixhD1ehz6nNWk
df = pd.read_csv("/content/spiral.csv")
df.head()
```

Downloading...

From: https://drive.google.com/uc?id=1dL0Pwh01o3k8p_hK633ixhD1ehz6nNWk

To: /content/spiral.csv

100% 12.9k/12.9k [00:00<00:00, 41.1MB/s]

	x1	x2	y
0	0.000000	0.000000	0
1	-0.000650	0.010080	0
2	0.009809	0.017661	0
3	0.007487	0.029364	0
4	-0.000027	0.040404	0

```
# Separating feature and label columns
X = df.iloc[:, :-1].to_numpy()
y = df.iloc[:, -1].to_numpy()
```

```

d = 2
n = 3
m = y.shape[0]
# initialize parameters randomly
W = 0.01 * np.random.randn(d,n)
b = np.zeros((1,n))

# compute scores for a linear classifier
z = np.dot(X, W) + b
print(z.shape) # should be mXn = #examples X #output

(300, 3)

```

Lets raise these scores to base e , and normalise them to get probabilities

```

exp_z = np.exp(z)
probs = exp_z / np.sum(exp_z, axis=1, keepdims=True) # explain why axis=1

probs.shape # probabilities of class

(300, 3)

```

```

dz = probs
dz[range(m),y] -= 1 # subtracting 1 from class where i == j as dz = pi - I
#dz = dz/m

```

$\frac{\partial z_i}{\partial w_i}$ is something we calculated earlier as well in LR, it will be equal to X.

Now, to calculate dW, we just need to multiply X with dz

```

X.shape # shape (m, d)

(300, 2)

# shape (m, n ) as we have m samples and each sample has 3 class probab.
dz.shape

(300, 3)

# dW = dZ . X

dW = np.dot(X.T, dz) # check dimensions (2, 300) x (300 , 3) => (2, 3)
dW.shape

(2, 3)

```

```

b.shape # shape (1, n)

```

(1, 3)

We know that,

$$z = wx + b$$

So,

- $\frac{\partial z}{\partial b} = \frac{\partial(wx+b)}{\partial b} = 1$
- as wx will be treated as constant when calculating partial derivative w.r.t b

Now,

For calculating db ,

$$db = dz \cdot \frac{\partial Z}{\partial b} = dz$$

✓ But why sum?

Since we are performing GD and not SGD,

- we'll take sum of average across all points
- we already took the average of dz ($dz = dz/m$)

Recall update equation of bias

$$b_{new} = b_{old} - \eta \times \frac{1}{m} \sum_{i=1}^m dz$$

```
db = np.sum(dz, axis=0, keepdims=True)
db.shape
```

(1, 3)

```
def backprop(probs, y):  
    # we know that dz = pi - I  
    dz = probs # dz = pi  
    dz[range(m),y] -= 1 # subtracting 1 where i ==j i.e. class label matches  
    dz = dz/m # taking average as we have m points  
    dW = np.dot(X.T, dz)  
    db = np.sum(dz, axis=0, keepdims=True)  
    return dW, db
```

Since, we are moving from right to left, to calculate the gradients, lets call it backpropagation

✓ Once, we have the gradients, we will update the parameters

```
lr = 0.1  
W += -lr * dW  
b += -lr * db
```

Lets put all this together in a class


```

max_iters = 500
lr = 1

d = X.shape[1]
n = len(np.unique(y))
m = X.shape[0]
W = 0.01 * np.random.randn(d,n)
b = np.zeros((1,n))
loss_history = []

for i in range(max_iters):
    # evaluate the class probs
    z = np.dot(X, W) + b
    exp_z = np.exp(z)
    probs = exp_z/np.sum(exp_z, axis=1, keepdims=True)

    # compute the loss: average cross-entropy loss and regularization
    error = -np.log(probs[range(m), y])
    loss = np.sum(error)/m
    loss_history.append(loss)
    if i % 100 == 0:
        print(f"iteration: {i}, loss: {loss}")

    # compute the gradient on score
    dZ = probs
    dZ[range(m),y] -= 1
    dZ = dZ/m
    dW = np.dot(X.T, dZ)
    db = np.sum(dZ, axis=0, keepdims=True)

    # perform a parameter update using gradient descent
    W += -lr * dW
    b += -lr * db

# history = pd.DataFrame({'step': list(range(max_iters)), 'loss': loss_history})
# history.plot(x='step', y='loss', xlabel='step', ylabel='loss')

iteration: 0, loss: 1.1034975631352493
iteration: 100, loss: 0.7447334923011109
iteration: 200, loss: 0.7405327174688755
iteration: 300, loss: 0.7402162294779646
iteration: 400, loss: 0.740185995247451

def predict(X):
    Z = np.dot(X, W) + b
    Z_e = np.exp(Z)
    probs = Z_e/np.sum(Z_e, axis=1, keepdims=True)
    return np.argmax(probs, axis=1)

print(f"Training Accuracy {np.sum(predict(X) == y)/m}")

Training Accuracy 0.5266666666666666

```

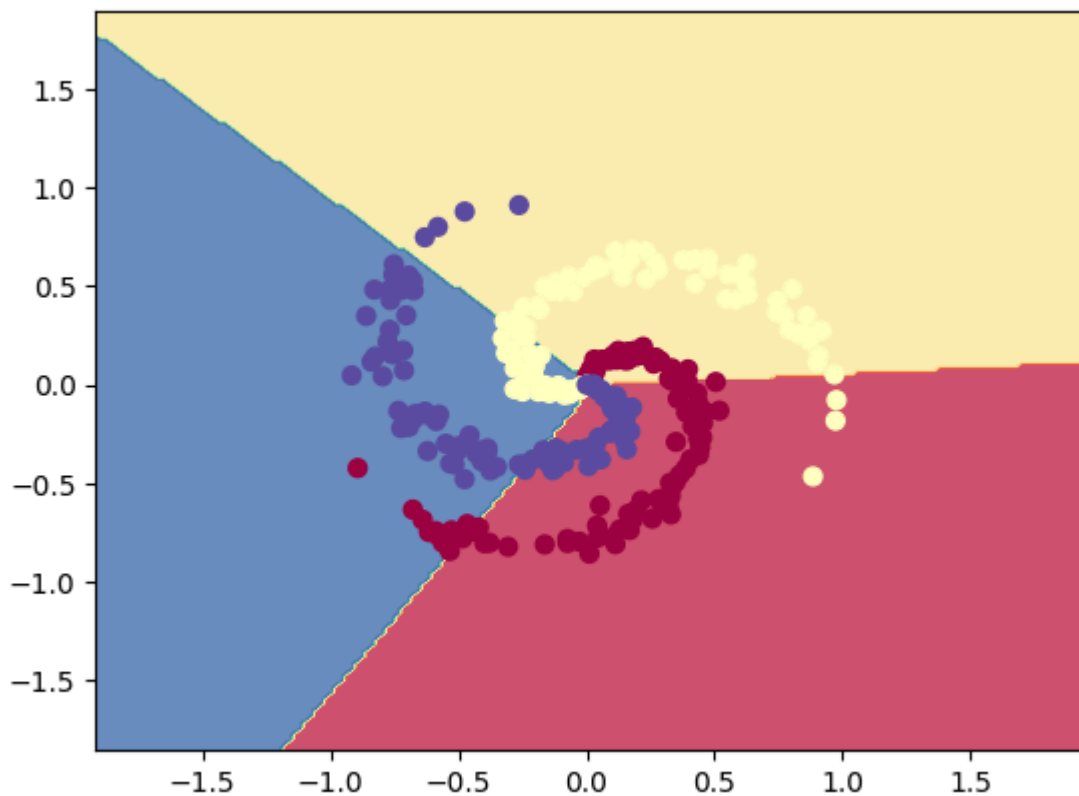
We have adapted the Logistic Regression model to work for mult-class setting.

Let's plot the decision boundaries

```
# create a 2D grid
step = 0.02
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, step), np.arange(y_min, y_max, step))

# predict for all the points in the grid
y_hat = predict(np.c_[xx.ravel(), yy.ravel()]) # concatenates along second axis
y_hat = y_hat.reshape(xx.shape)

# plot
fig = plt.figure()
plt.contourf(xx, yy, y_hat, cmap=plt.cm.Spectral, alpha=0.8)
plt.scatter(X[:, 0], X[:, 1], c=y, s=40, cmap=plt.cm.Spectral)
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.show()
```



- We can see that the model has learnt three decision boundaries
- But since haven't added any intermediate layer of neurons to create complex features, we don't expect the model to learn complex boundaries

✓ Softmax Classifier code wrapped in a Python class

```

class SoftmaxClassifier:
    def __init__(self, n_features, n_outputs):
        self.d = n_features
        self.n = n_outputs
        self.W = 0.01 * np.random.randn(d,n)
        self.b = np.zeros((1,n))
        self.loss = []

    def fwdprop(self, X):
        z = np.dot(X, self.W) + self.b
        exp_z = np.exp(z)
        probs = exp_z / np.sum(exp_z, axis=1, keepdims=True)
        return probs

    def cce_loss(self, probs, y):
        m = y.shape[0]
        error = -np.log(probs[range(m), y])
        return np.sum(error)/m

    def backprop(self, probs, y):
        m = y.shape[0]
        dz = probs
        dz[range(m),y] -= 1
        dz = dz/m
        dW = np.dot(X.T, dz)
        db = np.sum(dz, axis=0, keepdims=True)
        return dW, db

    def fit(self, X, y, lr=0.1, max_iters=50):

        for i in range(max_iters):
            # evaluate the class probs
            probs = self.fwdprop(X)

            # compute the loss: average cross-entropy loss and regularization
            loss = self.cce_loss(probs, y)

            # compute the gradient on score
            dW, db = self.backprop(probs, y)

            # perform a parameter update using gradient descent
            self.W += -lr * dW
            self.b += -lr * db
            self.loss.append(loss)

        self.history = pd.DataFrame({
            'step': list(range(max_iters)),
            'loss': self.loss})

    def plot_loss(self):
        return self.history.plot(x='step', y='loss',xlabel='step', ylabel='loss')

    def predict(self, X):
        probs = self.fwdprop(X)
        return np.argmax(probs, axis=1)

```

```

model = SoftmaxClassifier(n_features=2, n_outputs=3)
model.fit(X, y, lr=1, max_iters=500)
#model.plot_loss()
#print('training accuracy:', np.sum(model.predict(X) == y)/X.shape[0])

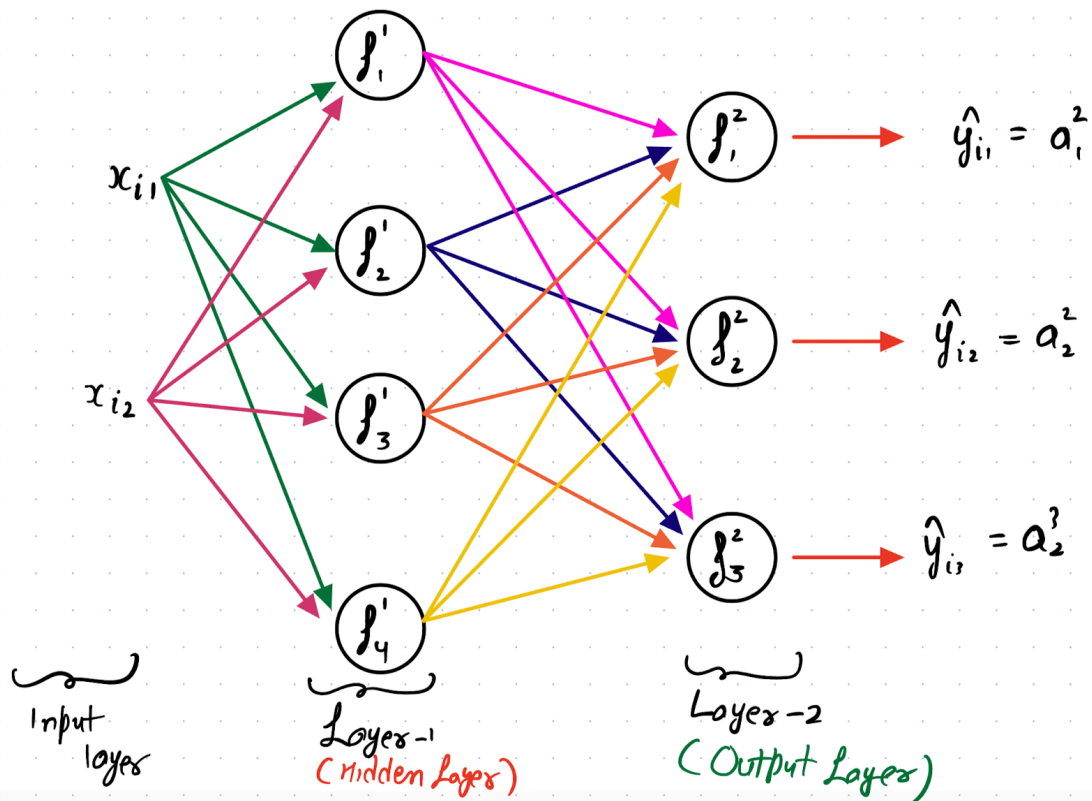
```

Assessments Covered (Misc):-

- Identify the Function: <https://www.scaler.com/hire/test/problem/16643/> (To be attempted after post read; contains mention of perceptron)

✓ MLP: Multi Layer Perceptron

We make our model/ network complex by adding a layer between our input and output.



Let's understand what we did here.

What are we given with?

- We know that we have 2 feature input and
- we have 3 classes in output.

Understanding the architecture

- Here, we use superscript notation to represent the layer.
 - So, instead of writing f_1 , we write it as f_1^1 i.e. neuron 1 of layer 1

Now,

We took our 2 inputs (Layer - 0 / **Input Layer**) and connected them with the the 4 neuron of **Layer-1**

- These 4 neurons will be connected with the 3 neuron of the **output layer** (Layer -2).

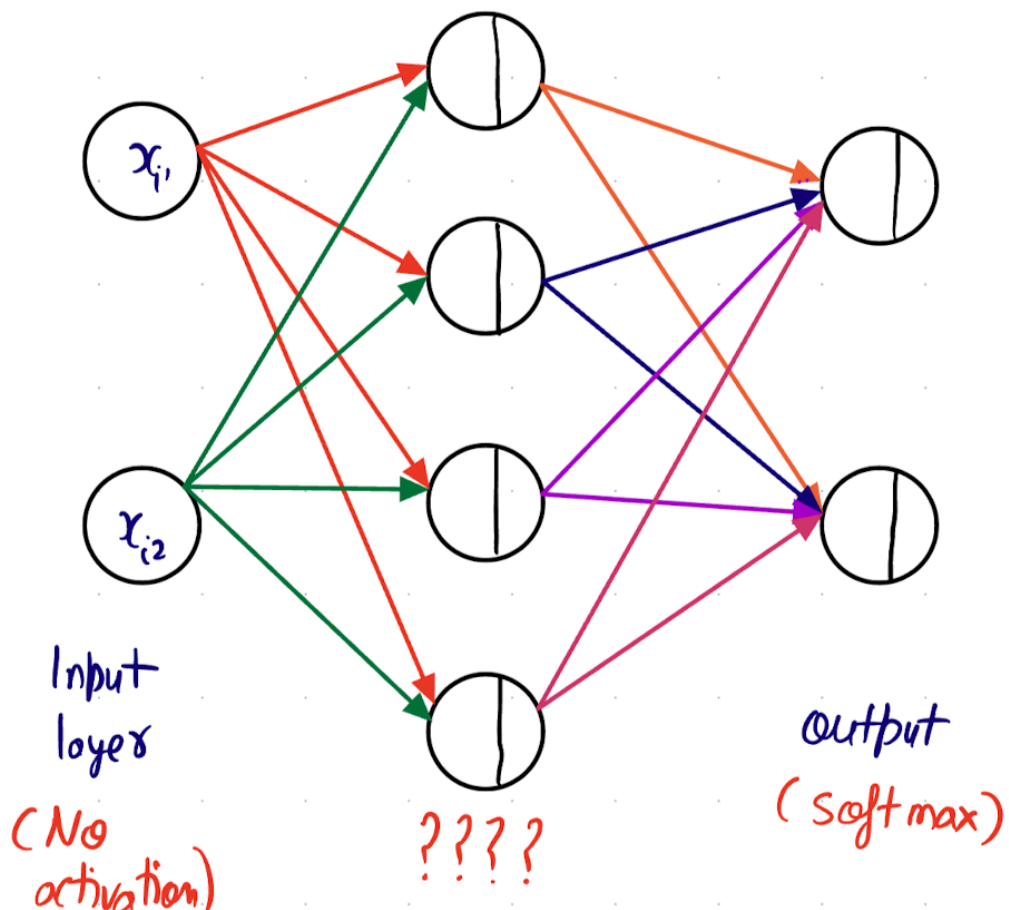
This intermediate layer in between the input and output layer is called **Hidden layer**

✓ But, why is called **hidden layer**?

As both input and output are **hidden** from this layer, and

- we don't directly deal with this layer,
- hence, it is called **hidden layer**.

We make our network complex by increasing the number of neuron in the layer or by increasing the number of layers.



Question: What will be the activation function for output layer?

Ans: Since it is a multiclass classification problem, we'll use softmax activation function

What will be the activation function for **hidden** layer ?

Recall that we want our model to learn non linear decision boundary,

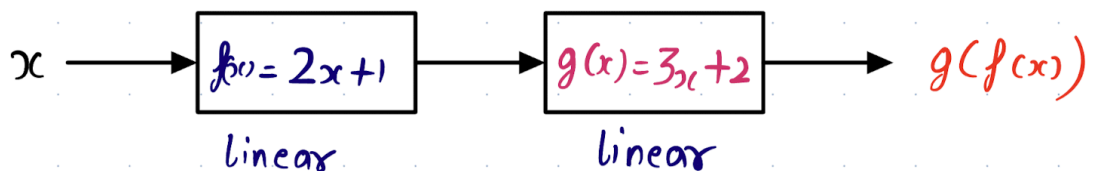
- Hence, we'll always use non linear activation function

✓ Question: What happens if each activation function in MLP is a linear function?

Let's try to understand it using an example

✓ **Case 1:**

Given $f(x) = 2x + 1$, $g(x) = 3x + 2$, will $f(g(x))$ be linear or non-linear?



$$\begin{aligned}
 g(f(x)) &= 3(2x + 1) + 2 \\
 &= 6x + 5 \quad \text{linear}
 \end{aligned}$$

$$f(g(x)) = 2(3x + 2) + 1 = 6x + 5 \rightarrow \text{linear}$$

- Composition of two linear function is linear.

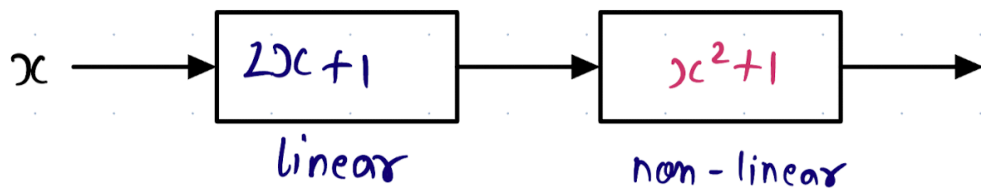
If we use linear activation

- it'll become a linear model.

Remember, we have to generate higher order non-linear features

✓ How non-linear function helps us here ?

Case 2: $f(x) = x^2 + 1$, $g(x) = 2x + 1$



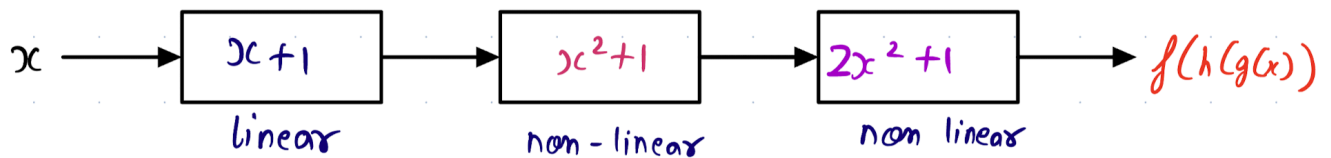
$$g(f(x)) = (2x+1)^2 + 1$$

$$= 4x^2 + 2x + 2 \quad \text{non-linear}$$

$$f(g(x)) = (2x + 1)^2 + 1 = 4x^2 + 2x + 2 \rightarrow \text{non-linear, higher order}$$

✓ Q. How about we stack non-linearity once more?

Case 3: $h(x) = x^2 + 1$, $f(x) = 2x^2 + 1$, $g(x) = x + 1$



$$f(h(g(x))) = 2[(x+1)^2 + 1]^2 + 1$$

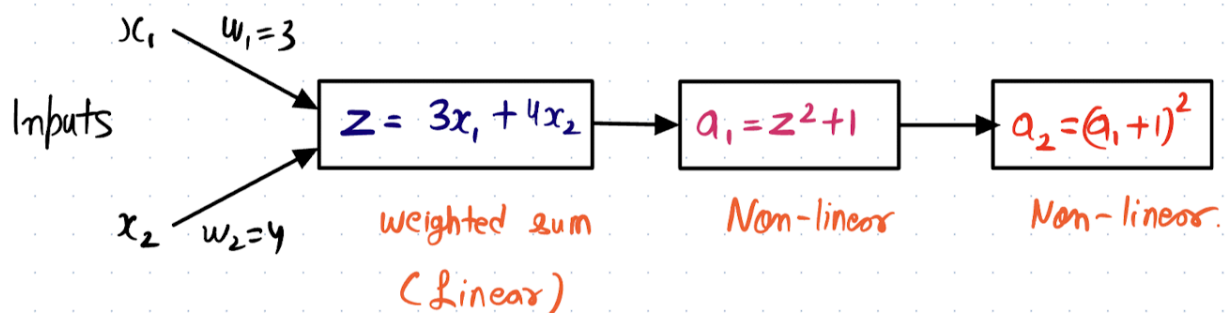
$$= 2[2x^2 + 2x + 3]^2 + 1 \quad \text{non-linear}$$

$f(h(g(x))) = 2((x+1)^2 + 1)^2 + 1 = 2(2x^2 + 2x + 3)^2 + 1 \rightarrow$ non-linear, much higher order

Conclusion - Stacking a non-linearity over a linear function, and repeating the process may create complex features

For example, suppose we have 2 inputs: x_1, x_2

They are going through the following **computation graph**.



- z is a weighted sum, so it'll be linear only: $z = 3x_1 + 4x_2$
- a_1 will be non-linear and complex: $a_1 = 9x_1^2 + 16x_2^2 + 24x_1x_2$
- a_2 will be non-linear and much more complex:
 $a_2 = (9x_1^2 + 16x_2^2 + 24x_1x_2 + 1)^2 = (9x_1^2 + 16x_2^2)^2 + 2(9x_1^2 + 16x_2^2)(24x_1x_2 + 1)$

What non-linear function shall we use? Have you seen any non linear function in the ML algos so far ?

Ans: Recall Sigmoid is one of the non linear activation function we saw.

Question: Should we use different activation functions in different layers?

Answer:

- Theoretically we can. But studies have shown that it is not of much value.

✓ Activation function

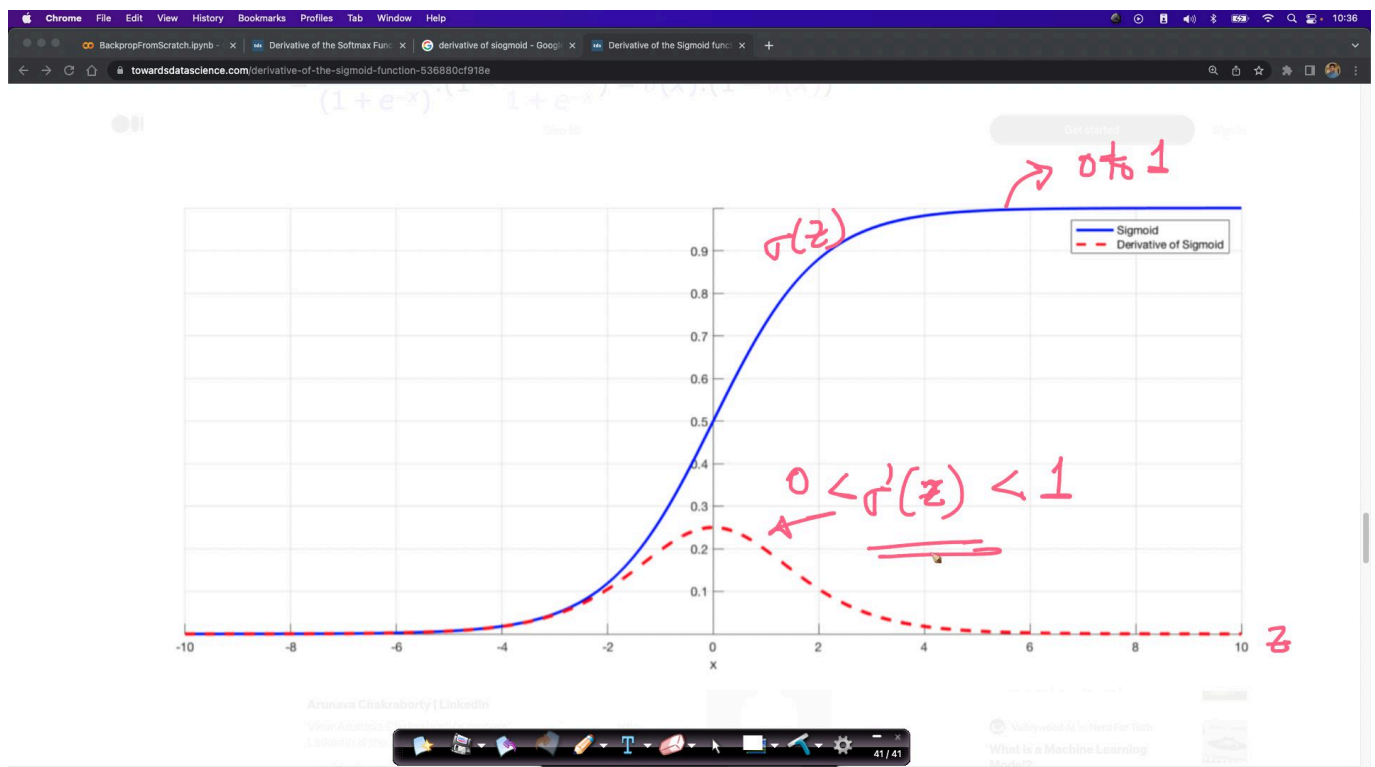
We saw that how sigmoid can be used as a activation function for **hidden layer**.

Do note that

- Domain of sigmoid is $(-\infty, \infty)$
- Range is $(0,1)$
- Derivative of sigmoid also lies between $(0,1)$

Also, derivative of sigmoid can also be represented in terms of sigmoid.

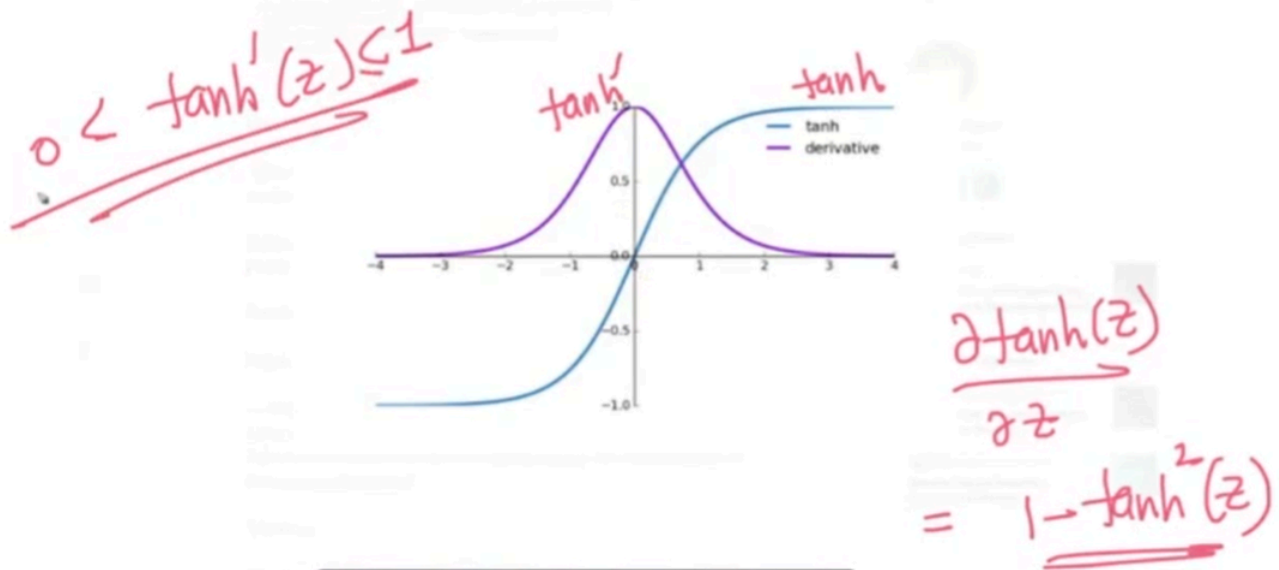
However, Sigmoid activation function was very popular in 1980's and 1990's



✓ Are there any other activation functions ?

tanh function

- Shifted version of sigmoid function
- Works better than sigmoid almost all the time - mean value is zero
- Inputs lies in the range: $(-\infty, \infty)$
- Output lies in the range: $(-1, 1)$
- We don't use tanh function very often, unless we want output to lie in the range of $(-1, 1)$.
- Formula: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- Derivative of tanh function: $\frac{d(\tanh(z))}{dz} = 1 - \tanh^2(z)$
 - This lies between $(0, 1)$



✓ Issue with Sigmoid and tanh

✓ Vanishing Gradients

Downside of both sigmoid and tanh is that their gradient is <1 , for most of the values of z

- This hampers the gradient descent process, and the calculated gradients will be very small.

✓ Why does small gradient hampers GD process ?

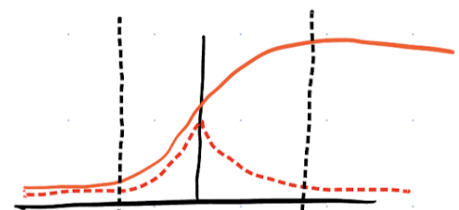
Since the activation functions are sigmoid or tanh

- We know that derivative of these functions lie between (0, 1).
- So, the product of these terms inside the bracket, will become very small.
- In fact, as the number of layers in the NN increase, this product will become smaller and smaller.

For σ or $\tanh$

$$\frac{\partial J}{\partial w} = \underbrace{\frac{\partial J}{\partial p}}_{(0,1)} \cdot \underbrace{\frac{\partial p}{\partial z}}_{(0,1)} \cdot \underbrace{\frac{\partial z}{\partial w}}_{(0,1)}$$

$\Rightarrow$ All derivatives $0 < \delta < 1$



For most values of z
 $\delta \rightarrow 0$

The equation to update the weight w is: $w_{new} = w_{old} - \eta \cdot \left. \frac{\partial loss}{\partial w} \right|_{w_{old}}$

- We just saw that this partial derivative value becomes miniscule, as the number of layers increase.
- As a result, the NN gets trained very very slowly.
- In fact, for close to 2 decades, people could not imagine using a NN with more than 3-4 layers.

$$w_{\text{new}} = w_{\text{old}} - \underbrace{\frac{\partial L}{\partial w}}_{\text{small value}} \bigg|_{w_{\text{old}}}$$

$w_{\text{new}} \approx w_{\text{old}}$
i.e. very small update.

- ✓ How shall we deal with this problem ?
- ✓ Question: What should be the properties of ideal activation function?

It should be:

- differentiable
- non linear
- easy to calculate
- gradient ≥ 1 for big range of z

✓ ReLu

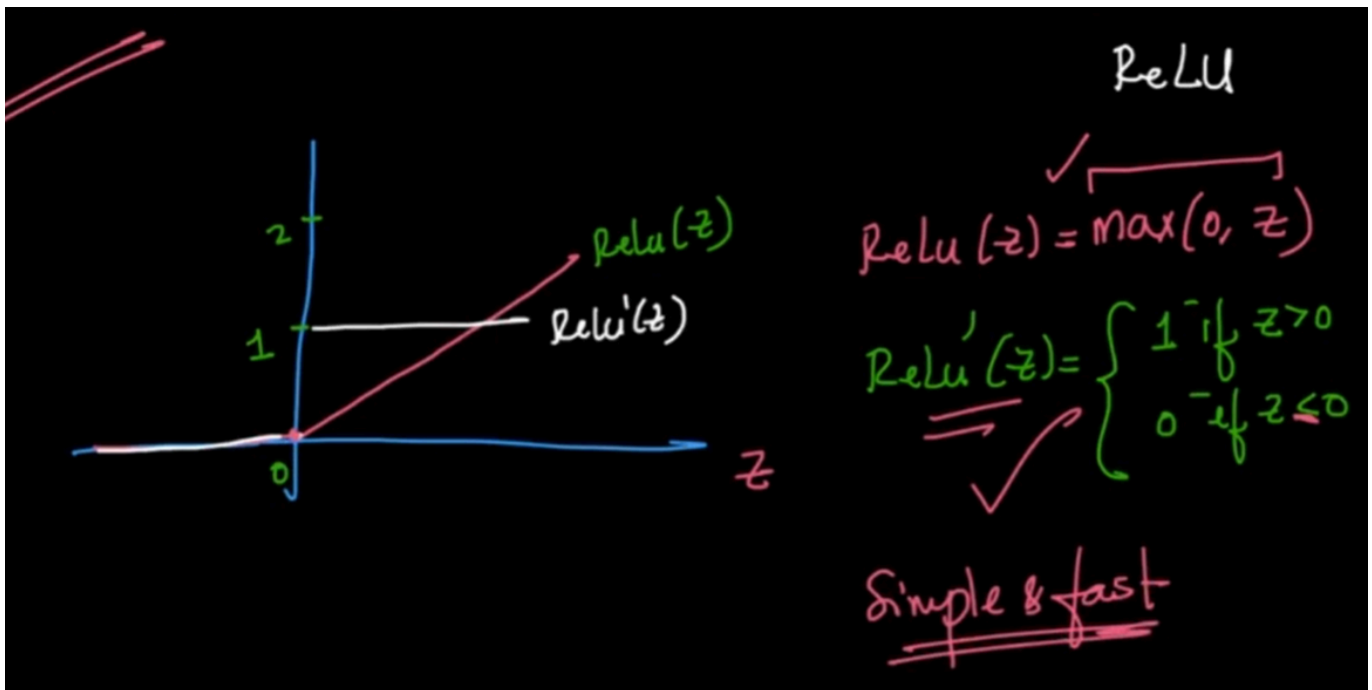
- Stands for **Rectified Linear Unit**.
- It says, that for any positive value, it will return that value as it is: $Relu(z) = z$, if $z > 0$
- Otherwise, it returns 0: $Relu(z) = 0$, if $z \leq 0$

It can be stated as: $Relu(z) = \max(z, 0)$

- Very fast to compute.
- Derivative of ReLu wrt z : $Relu'(z) = \begin{cases} 1, & \text{if } z > 0 \\ 0, & \text{if } z < 0 \end{cases}$

- Though it is not differential at 0, as it is not continuous
- So here we take an approximation, for it to work. So we make it as:

$$ReLU'(z) = \begin{cases} 1, & \text{if } z > 0 \\ 0, & \text{if } z \leq 0 \end{cases}$$

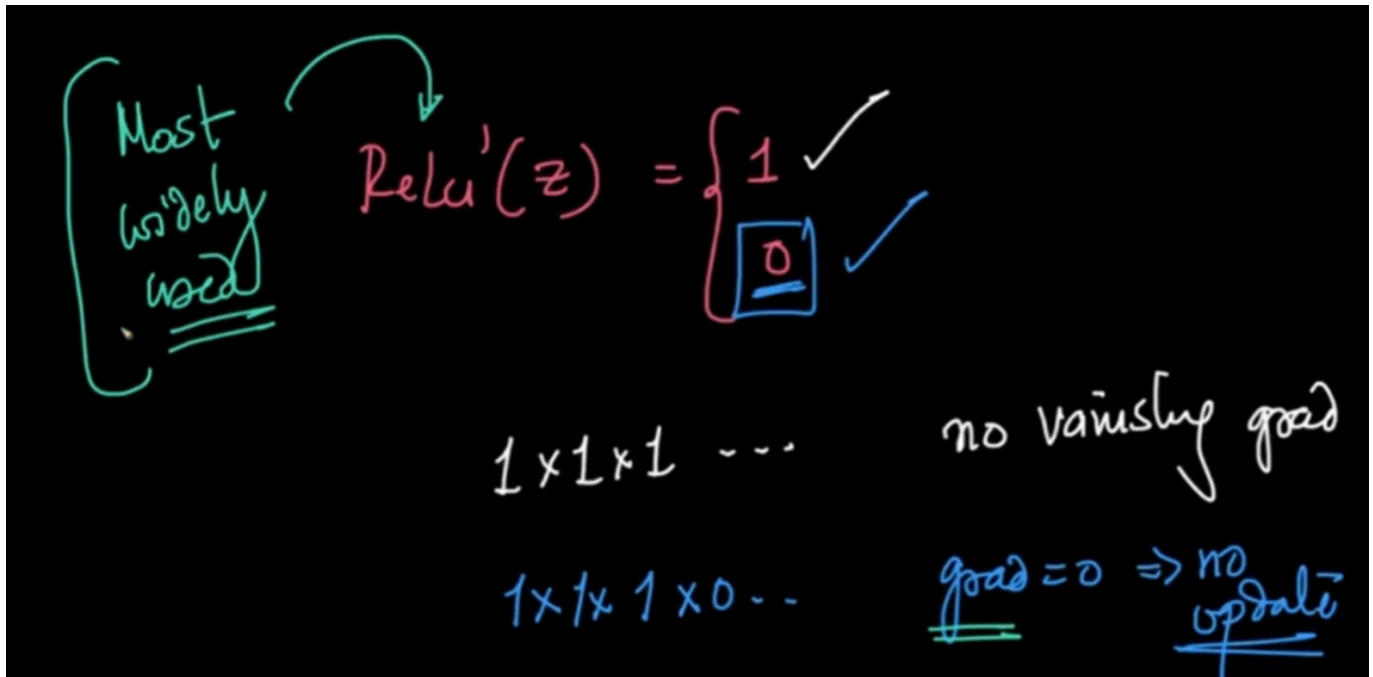


✓ Question: Is there a problem in practically using ReLU as activation function?

Yes. Even though it is the most widely used activation function in the world of Deep Learning, there is a slight problem.

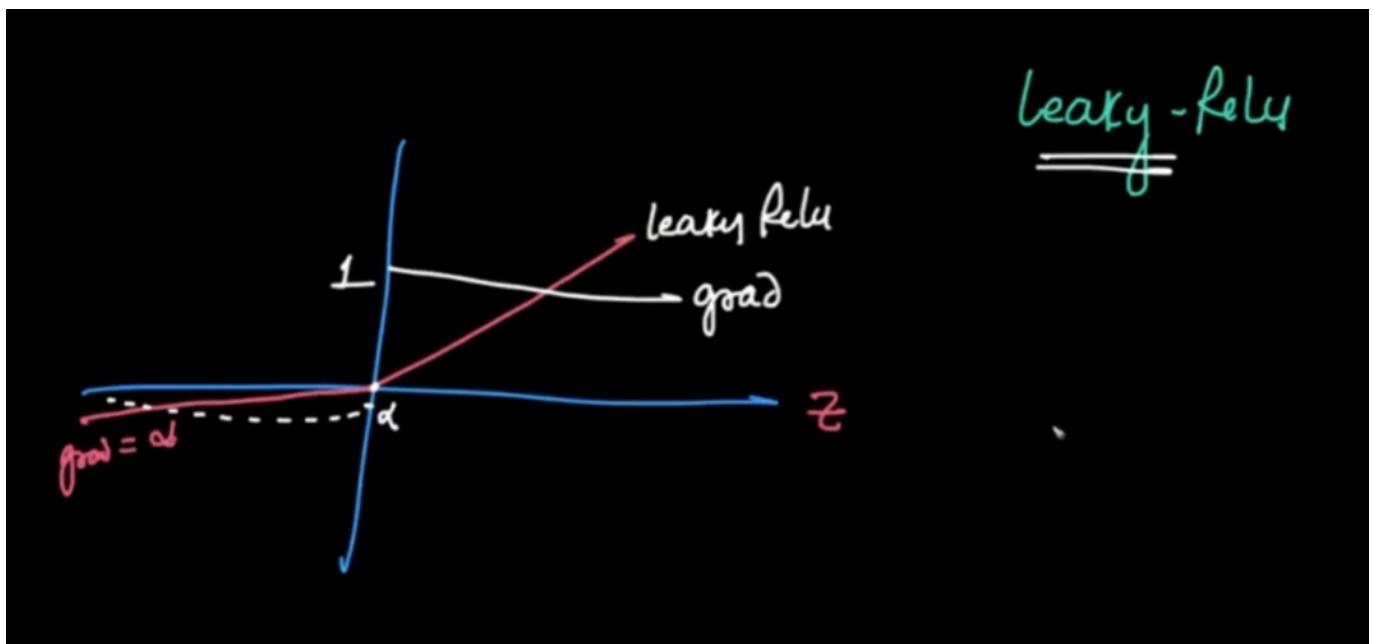
- If even one derivative term in calculation of $\frac{\partial Loss}{\partial w}$ gets the value as 0, the entire term will become zero.
- Hence, there is no update in the value of weight.
- This is also known as "**dying ReLU**"
- So there's a potential **vanishing gradient** problem.
 - While calculating backprop, if one of the derivative is 0, the whole update will become zero and network won't update

However, to deal with this, a slight modification is made to ReLU, and we get another activation function known as **Leaky ReLU**



✓ Leaky ReLU

- This is very similar to ReLU but there's a twist
- In case of negative values, we add a small gradient (α) associated with it, instead of having 0.
- Gradient: $\text{Leaky ReLU}'(z) = \begin{cases} 1, & \text{if } z > 0 \\ \alpha, & \text{if } z \leq 0 \end{cases}$



✓ Notations for N layer NN

So, we introduce layer number in notation as superscript

Now, As layer increase, earlier weight notation will not work.

- So, we add layer number to weight notation as well.

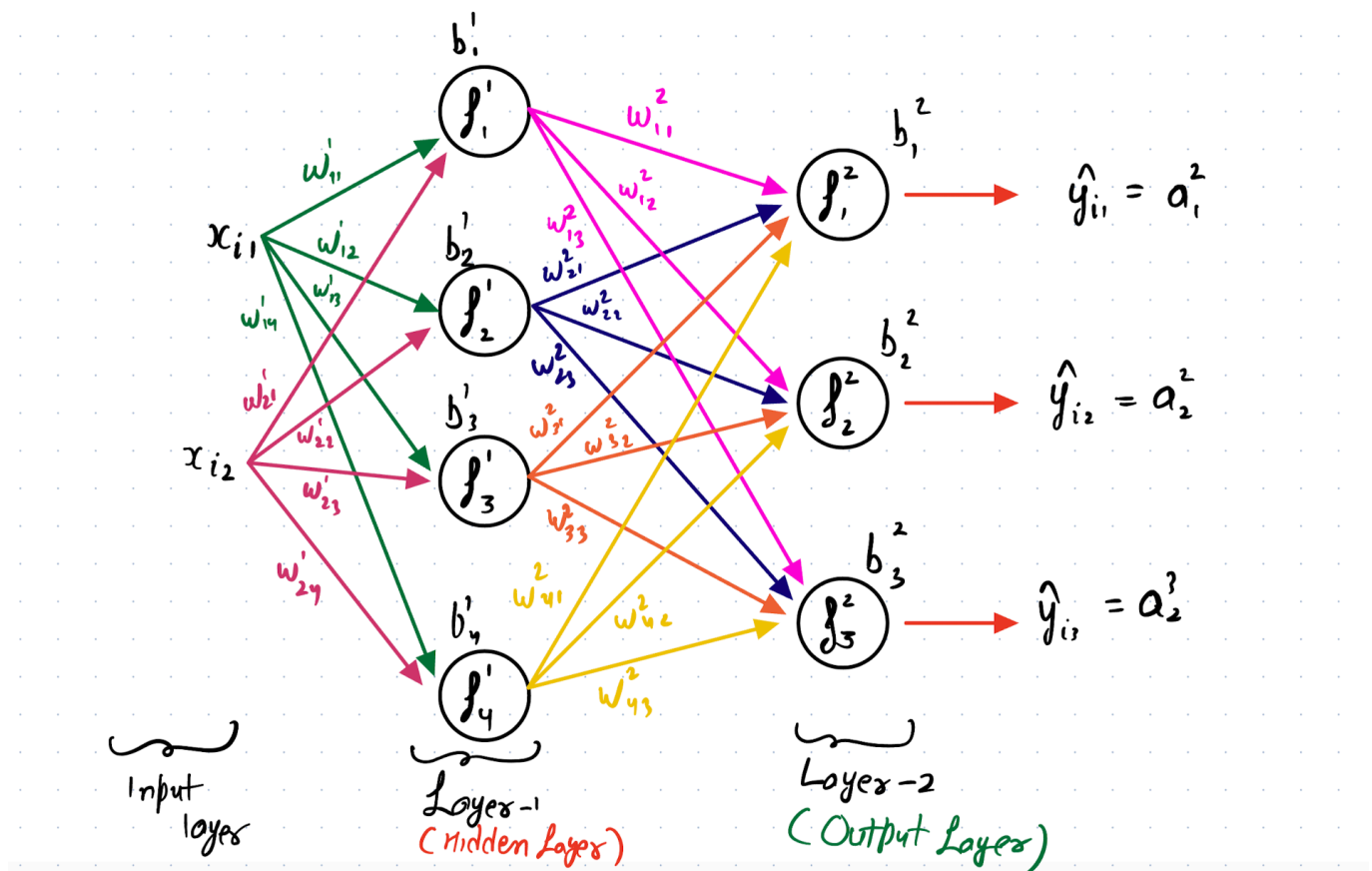
For weight: w_{ij}^L

where

- L represent layer number
- and i represents from neuron
- j represent to neuron

Bias: b_i^L where

- L represents the layer
- i represents the neuron.



Quiz

How many total parameters do we have for this network?

- 15
- 21
- 27

d. 30

Ans: 27 (c)

- For first layer,
 - we have $2 \times 4 = 8$ weights and 4 biases i.e. 12 params
- For second layer,
 - we have $4 \times 3 = 12$ and 3 biases i.e. 15 params

Hence, we have total of $12 + 15 = 27$

We will now have two sets of weights and biases (for the first and second layers) - W^1, W^2, b^1, b^2

What will be the dimensions of the parameters W^1, b^1, W^2, b^2 ?

Forget about the second layer for now, let's just focus on the first layer, and it's inputs.

W^1

- Here, we have **2 inputs** being passed to **4 neurons**
- Recall that each pair of input and neuron, has a unique weights value.
- Therefore, W^1 will be 2×4 matrix: $W^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 & w_{14}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 & w_{24}^1 \end{bmatrix}_{2 \times 4}$